# 데이터베이스
# (Oracle)

# 1. Oracle Database 설정

- **Oracle 설치**

1) http://www.oracle.com 오라클 홈페이지에서 회원가입 한다.
2) Oracle Database Express Edition 18c를 다운로드하여 설치한다.
3) 설치 시 데이터베이스 암호를 1234로 입력한다.

- **서비스 수동으로 변경**

1) 시작 -> 설정-> 제어판-> 관리도구 -> 서비스에서 Oracle과 관련된 Service를 찾아본다.
2) OracleServiceXE의 속성에서 시작유형을 자동에서 수동으로 변경한다.
3) OracleXETNSLinstener의 속성에서 시작유형을 자동에서 수동으로 변경한다.

- **Oracle Database 서버접속**

1) 시작메뉴 -> Oracle-OraDB18Home1-> SQL Plus를 실행한다.

```
SQL> CONNECT;
Enter user-name: system
Enter password: 1234
SQL> DISCONNECT;
```

2) 비밀번호 변경 시 sysdba로 로그인 하여 비밀번호를 변경한다.

```
1) CONNECT
2) Enter user-name: /AS SYSDBA
3) pass: 그냥 엔터
4) ALTER USER system IDENTIFIED BY newpass(새 비밀번호);
```

3) 오라클 SID(Session Identification)을 확인한다.

```
SQL> SELECT NAME FROM v$database;
```

- **데이터베이스/스키마/사용자 생성하기**

전체를 관리하는 최상위 데이터베이스를 CDB(Container DB), CDB 안에 들어있는 개발 독립 데이터베이스가 PDB(Pluggable DB)이다.

방법1. C## 접두사 사용(권장 표준 방식): CDB에 공통 사용자를 생성하는 기본 방식이다. 계정명 앞에 C##을 붙여 생성한다.

```
CREATE USER c##username IDENTIFIED BY password;
```

방법2. C## 접두사 없이 생성: _ORACLE_SCRIPT 세션을 TRUE로 변경하여 오라클 12c 이상에서 옛날 방식으로 계정을 생성한다.

```
ALTER SESSION SET "_ORACLE_SCRIPT" = TRUE;
CREATE USER username IDENTIFIED BY password;
```

방법 3. PDB로 이동 후 생성 (운영 환경 추천)

```
SHOW CON_NAME; -- 현재 데이터베이스 컨테이너 확인
SHOW PDBS; -- 사용할 PDB 데이터베이스 컨테이너 목록 확인
ALTER SESSION SET CONTAINER = XEPDB1; -- 특정 PDB 데이터베이스 컨테이너로 세션 변경 (예: XEPDB1)
CREATE USER username IDENTIFIED BY password; -- 새로운 사용자 생성
```

- **사용자 권한부여/비밀번호 변경/사용자 삭제**

```
GRANT CONNECT, RESOURCE, DBA TO username; -- 사용자 권한 부여
SELECT USERNAME FROM DBA_USERS; -- 사용자 목록으로 생성된 사용자 확인
ALTER USER username IDENTIFIED BY newpassword; -- 사용자 비밀번호 변경
DROP USER username CASCADE; -- 사용자 및 소요한 모든 객체 삭제
```

# 2. 학사관리 데이터베이스

- **PROFESSOR(교수) 테이블의 구조**

| 열이름 | 데이터형 | 정보내용 |
|---|---|---|
| PCODE | CHAR(3) | 교수번호 (PK, NOT NULL) |
| PNAME | VARCHAR(15) | 교수이름 (NOT NULL) |
| DEPT | VARCHAR(30) | 소속학과 (NOT NULL) |
| HIREDATE | DATE | 임용일자 (DEFAULT SYSDATE) |
| TITLE | VARCHAR(15) | 직급 |
| SALARY | INT | 급여 (DEFAULT 0) |

- **STUDENTS(학생) 테이블의 구조**

| 열이름 | 데이터형 | 정보내용 |
|---|---|---|
| SCODE | CHAR(8) | 학생번호 (PK, NOT NULL) |
| SNAME | VARCHAR(15) | 학생이름 (NOT NULL) |
| DEPT | VARCHAR(30) | 소속학과 (NOT NULL) |
| YEAR | INT | 학년 (DEFAULT 1) |
| BIRTHDAY | DATE | 생년월일 |
| ADVISOR | CHAR(3) | 지도교수 (FK) |

- **COURSE(강좌) 테이블의 구조**

| 열이름 | 데이터형 | 정보내용 |
|---|---|---|
| LCODE | CHAR(4) | 강좌번호(PK, NOT NULL) |
| LNAME | VARCHAR(50) | 강좌이름 (NOT NULL) |
| HOURS | INT | 강의시수 (DEFAULT 3) |
| ROOM | CHAR(3) | 강의실 |
| INSTRUCTOR | CHAR(3) | 담당교수(FK) |
| CAPACITY | INT | 최대수강인원수 (DEFAULT 0) |
| PERSONS | INT | 수강신청인원수 (DEFAULT 0) |

- **ENROLLMENT(수강신청) 테이블의 구조**

| 열이름 | 데이터형 | 정보내용 |
|---|---|---|
| LCODE | CHAR(4) | 강좌번호 |
| SCODE | CHAR(8) | 학생번호 |
| EDATE | DATE | 수강신청일 |
| GRADE | INT | 성적 |

- **ERD 다이어그램 (Oracle)**

```
[파일]-[Data Modeler]-[임포트]-[데이터 딕션너리]-[접속 이름 선택]-[스카마/데이터베이스 선택]-[다음]-[테이블/객체 선택]-[다음]-[완료]-[병합]
```

- **ERD 다이어그램 (MySQL)**

```
[Database]-[Reverse Engineer Database]
```

- **PROFESSOR(교수) 테이블의 데이터**

| PCODE(교수번호) | PNAME(교수이름) | DEPT(소속 학과) | HIREDATE(임용일) | TITLE(직급) | SALARY(급여) |
|---|---|---|---|---|---|
| 221 | 이병렬 | 전산 | 75/04/03 | 정교수 | 3,000,000 |
| 228 | 이재광 | 전산 | 91/09/19 | 부교수 | 2,500,000 |
| 311 | 강승일 | 전자 | 94/06/09 | 부교수 | 2,300,000 |
| 509 | 오문환 | 건축 | 92/10/14 | 조교수 | 2,000,000 |

- **STUDENT(학생) 테이블의 데이터**

| SCODE(학생번호) | SNAME(학생이름) | DEPT(소속학과) | YEAR(학년) | BIRTHDAY(생년월일) | ADVISOR(지도교수 번호) |
|---|---|---|---|---|---|
| 92414029 | 서연우 | 전산 | 3 | 73/10/06 | 228 |
| 92414033 | 김창덕 | 전산 | 4 | 73/10/26 | 221 |
| 92514009 | 이지행 | 전자 | 4 | 73/11/16 | 311 |
| 92514023 | 김형명 | 전자 | 4 | 73/08/29 | 311 |
| 92454018 | 이원구 | 건축 | 3 | 74/09/30 | 509 |
| 95454003 | 이재영 | 건축 | 4 | 76/02/06 | 509 |
| 95414058 | 박혜경 | 전산 | 4 | 76/03/12 | 221 |
| 96414404 | 김수정 | 전산 | 3 | 77/12/22 | 228 |

- **COURSE(강좌) 테이블의 데이터**

| LCODE (강좌번호) | LNAME (강좌이름) | HOURS (강의시간수) | ROOM (강의실) | INSTRUCTOR (담당교수번호) | CAPACITY (최대수강인원수) | PERSONS (수강신청인원수) |
|---|---|---|---|---|---|---|
| C301 | 파일처리론 | 3 | 506 | 221 | 100 | 80 |
| C401 | 데이터베이스 | 3 | 414 | 221 | 80 | 80 |
| C421 | 알고리즘 | 3 | 510 | 228 | 80 | 72 |
| C312 | 자료구조 | 2 | 510 | 228 | 100 | 60 |
| E221 | 논리회로 | 3 | 304 | 311 | 100 | 80 |
| A109 | 한국의건축문화 | 2 | 101 | 509 | 120 | 36 |

- **ENROLLMENT(수강신청) 테이블의 데이터**

| LCODE(강좌번호) | SCODE(학생번호) | EDATE(수강신청일) | GRADE(성적) |
|---|---|---|---|
| C401 | 92414033 | 98/03/02 | 85 |
| C301 | 92414033 | 98/03/02 | 80 |
| C421 | 92414033 | 98/03/02 | 0 |
| C401 | 95414058 | 98/03/03 | 90 |
| C312 | 95414058 | 98/03/03 | 80 |
| C401 | 92514023 | 98/03/03 | 70 |
| C301 | 92414029 | 98/03/03 | 90 |
| C421 | 92414029 | 98/03/03 | 0 |
| C312 | 92414029 | 98/03/03 | 70 |

- **학사관리 테이블 생성**

**테이블 관련 SQL**

```
-- 테이블구조
DESC 테이블명

-- 사용자 테이블 검색
SELECT TABLE_NAME FROM USER_TABLES;

-- 테이블 삭제
DROP TABLE 테이블명
```

---

**교수 테이블 생성**

```
CREATE TABLE PROFESSOR(
    PCODE           CHAR(3) NOT NULL,
    PNAME           VARCHAR(15) NOT NULL,
    DEPT            VARCHAR(30) NOT NULL,
    HIREDATE        DATE DEFAULT SYSDATE,
    TITLE           VARCHAR(15),
    SALARY          INT DEFAULT 0,
    PRIMARY KEY(PCODE)
);
```

---

**학생 테이블 생성**

```
CREATE TABLE STUDENT(
    SCODE           CHAR(8) NOT NULL,
    SNAME           VARCHAR(15) NOT NULL,
    DEPT            VARCHAR(30) NOT NULL,
    YEAR            INT DEFAULT 1,
    BIRTHDAY        DATE,
    ADVISOR         CHAR(3),
    PRIMARY KEY(SCODE),
    FOREIGN KEY(ADVISOR) REFERENCES PROFESSOR(PCODE)
);
```

---

**강좌 테이블생성**

```
CREATE TABLE COURSE(
    LCODE           CHAR(4) NOT NULL,
    LNAME           VARCHAR(50) NOT NULL,
    HOURS           INT DEFAULT 3,
    ROOM            CHAR(3),
    INSTRUCTOR      CHAR(3),
    CAPACITY        INT DEFAULT 0,
    PERSONS         INT DEFAULT 0,
    PRIMARY KEY(LCODE),
    FOREIGN KEY(INSTRUCTOR) REFERENCES PROFESSOR(PCODE)
);
```

---

**수강신청 테이블 생성**

```
CREATE TABLE ENROLLMENT(
     LCODE CHAR(4) NOT NULL,
     SCODE CHAR(8) NOT NULL,
     EDATE DATE,
     GRADE INT DEFAULT 0,
     PRIMARY KEY(LCODE, SCODE),
     FOREIGN KEY(LCODE) REFERENCES COURSE(LCODE),
     FOREIGN KEY(SCODE) REFERENCES STUDENT(SCODE)
);
```

---

- **학사관리 데이터 입력**

**교수 테이블 데이터입력**

```
INSERT INTO PROFESSOR(PCODE,PNAME,DEPT,HIREDATE,TITLE,SALARY) VALUES('221','이병렬','전산','75/04/03','정교수',3000000);
INSERT INTO PROFESSOR(PCODE,PNAME,DEPT,HIREDATE,TITLE,SALARY) VALUES('228','이재광','전산','91/09/19','부교수',2500000);
INSERT INTO PROFESSOR(PCODE,PNAME,DEPT,HIREDATE,TITLE,SALARY) VALUES('311','강승일','전자','94/06/09','부교수',2300000);
INSERT INTO PROFESSOR(PCODE,PNAME,DEPT,HIREDATE,TITLE,SALARY) VALUES('509','오문환','건축','92/10/14','조교수',2000000);
```

**학생 테이블 데이터 입력**

```
INSERT INTO STUDENT(SCODE,SNAME,DEPT,YEAR,BIRTHDAY,ADVISOR) VALUES('92414029','서연우','전산',4,'73/10/06','228');
INSERT INTO STUDENT(SCODE,SNAME,DEPT,YEAR,BIRTHDAY,ADVISOR) VALUES('92414033','김창덕','전산',4,'73/10/26','221');
INSERT INTO STUDENT(SCODE,SNAME,DEPT,YEAR,BIRTHDAY,ADVISOR) VALUES('92514009','이지행','전자',4,'73/11/16','311');
INSERT INTO STUDENT(SCODE,SNAME,DEPT,YEAR,BIRTHDAY,ADVISOR) VALUES('92514023','김형명','전자',4,'73/08/29','311');
INSERT INTO STUDENT(SCODE,SNAME,DEPT,YEAR,BIRTHDAY,ADVISOR) VALUES('92454018','이원구','건축',4,'74/09/30','509');
INSERT INTO STUDENT(SCODE,SNAME,DEPT,YEAR,BIRTHDAY,ADVISOR) VALUES('95454003','이재영','건축',3,'76/02/06','509');
INSERT INTO STUDENT(SCODE,SNAME,DEPT,YEAR,BIRTHDAY,ADVISOR) VALUES('95414058','박혜경','전산',3,'76/03/12','221');
INSERT INTO STUDENT(SCODE,SNAME,DEPT,YEAR,BIRTHDAY,ADVISOR) VALUES('96414404','김수정','전산',3,'77/12/22','228');
```

**강좌테이블 데이터 입력**

```
INSERT INTO COURSE(LCODE, LNAME, HOURS, ROOM, INSTRUCTOR, CAPACITY, PERSONS)
VALUES('C301', '파일처리론', 3 ,'506', '221', 100, 80);
INSERT INTO COURSE(LCODE, LNAME, HOURS, ROOM, INSTRUCTOR, CAPACITY, PERSONS)
VALUES('C401', '데이터베이스', 3, '414', '221', 80, 80);
INSERT INTO COURSE(LCODE, LNAME, HOURS, ROOM, INSTRUCTOR, CAPACITY, PERSONS)
VALUES('C421', '알고리즘', 3, '510', '228', 80, 72);
INSERT INTO COURSE(LCODE, LNAME, HOURS, ROOM, INSTRUCTOR, CAPACITY, PERSONS)
VALUES('C312', '자료구조', 2, '510', '228', 100, 60);
INSERT INTO COURSE(LCODE, LNAME, HOURS, ROOM, INSTRUCTOR, CAPACITY, PERSONS)
VALUES('E221','논리회로', 3, '304', '311', 100, 80);
INSERT INTO COURSE(LCODE, LNAME, HOURS, ROOM, INSTRUCTOR, CAPACITY, PERSONS)
VALUES('A109','한국의건축문화', 2, '101', '509', 120, 36);
```

**수강신청 테이블 데이터 입력**

```
INSERT INTO ENROLLMENT(LCODE, SCODE, EDATE, GRADE) VALUES('C401', '92414033', '98/03/02', 85);
INSERT INTO ENROLLMENT(LCODE, SCODE, EDATE, GRADE) VALUES('C301', '92414033', '98/03/02', 80);
INSERT INTO ENROLLMENT(LCODE, SCODE, EDATE, GRADE) VALUES('C421', '92414033', '98/03/02', 0);
INSERT INTO ENROLLMENT(LCODE, SCODE, EDATE, GRADE) VALUES('C401', '95414058', '98/03/03', 90);
INSERT INTO ENROLLMENT(LCODE, SCODE, EDATE, GRADE) VALUES('C301', '95414058', '98/03/03', 80);
INSERT INTO ENROLLMENT(LCODE, SCODE, EDATE, GRADE) VALUES('C312', '95414058', '98/03/03', 80);
INSERT INTO ENROLLMENT(LCODE, SCODE, EDATE, GRADE) VALUES('C401', '92514023', '98/03/03', 70);
INSERT INTO ENROLLMENT(LCODE, SCODE, EDATE, GRADE) VALUES('C301', '92514023', '98/03/03', 70);
INSERT INTO ENROLLMENT(LCODE, SCODE, EDATE, GRADE) VALUES('C421', '92514023', '98/03/03', 70);
INSERT INTO ENROLLMENT(LCODE, SCODE, EDATE, GRADE) VALUES('C301', '92414029', '98/03/03', 90);
INSERT INTO ENROLLMENT(LCODE, SCODE, EDATE, GRADE) VALUES('C421', '92414029', '98/03/03', 0);
INSERT INTO ENROLLMENT(LCODE, SCODE, EDATE, GRADE) VALUES('C312', '92414029', '98/03/03', 70);
INSERT INTO ENROLLMENT(LCODE, SCODE, EDATE, GRADE) VALUES('E221', '92414029', '98/03/03', 75);
INSERT INTO ENROLLMENT(LCODE, SCODE, EDATE, GRADE) VALUES('A109', '92414029', '98/03/03', 90);
INSERT INTO ENROLLMENT(LCODE, SCODE, EDATE, GRADE) VALUES('C301', '92514009', '98/03/03', 70);
INSERT INTO ENROLLMENT(LCODE, SCODE, EDATE, GRADE) VALUES('C401', '92514009', '98/03/03', 85);
INSERT INTO ENROLLMENT(LCODE, SCODE, EDATE, GRADE) VALUES('E221', '92514009', '98/03/03', 85);
INSERT INTO ENROLLMENT(LCODE, SCODE, EDATE, GRADE) VALUES('C301', '96414404', '98/03/04', 75);
INSERT INTO ENROLLMENT(LCODE, SCODE, EDATE, GRADE) VALUES('C401', '96414404', '98/03/04', 75);
INSERT INTO ENROLLMENT(LCODE, SCODE, EDATE, GRADE) VALUES('C421', '96414404', '98/03/04', 75);
INSERT INTO ENROLLMENT(LCODE, SCODE, EDATE, GRADE) VALUES('C312', '92454018', '98/03/04', 90);
INSERT INTO ENROLLMENT(LCODE, SCODE, EDATE, GRADE) VALUES('E221', '92454018', '98/03/04', 90);
INSERT INTO ENROLLMENT(LCODE, SCODE, EDATE, GRADE) VALUES('A109',' 95454003', '98/03/05', 85);
INSERT INTO ENROLLMENT(LCODE, SCODE, EDATE, GRADE) VALUES('C301', '95454003', '98/03/05', 83);
INSERT INTO ENROLLMENT(LCODE, SCODE, EDATE, GRADE) VALUES('E221', '95454003', '98/03/05', 75);
```

- **학사관리 데이터베이스 조회**

**1) 교수 테이블의 모든 행, 모든 칼럼의 데이터를 검색한다.**

```
SELECT * FROM PROFESSOR;
```

**2) 교수 테이블에서 모든 교수의 이름, 학과, 직급 칼럼을 검색한다.**

```
SELECT PNAME, DEPT, TITLE FROM PROFESSOR;
```

**3) 교수 테이블에서 교수들이 근무하는 소속 학과를 중복을 제거하여 검색한다.**

```
SELECT DISTINCT(DEPT) FROM PROFESSOR;
```

**4) 학생 테이블에서 '전산'학과 이면서 3학년 학생들의 모든 칼럼을 검색한다.**

```
SELECT * FROM STUDENT WHERE DEPT='전산' AND YEAR=3;
```

**5) 학생 테이블에서 3학년이거나 4학년 학생들의 모든 칼럼을 검색한다.**

```
SELECT * FROM STUDENT WHERE YEAR=3 OR YEAR=4;
```

```
SELECT * FROM STUDENT WHERE YEAR IN (3, 4);
```

**6) 학생 테이블에서 생년월일이 '1973-01-01'에서 '1973-12-31' 사이의 학생들의 모든 칼럼을 검색한다.**

```
SELECT * FROM STUDENT
WHERE BIRTHDAY >= '1973-01-01' AND BIRTHDAY <= '1973-12-31';
```

```
SELECT * FROM STUDENT
WHERE  BIRTHDAY BETWEEN '1973-01-01' AND '1973-12-31';
```

**7) 학생 테이블에서 모든 칼럼과 생년월일의 포맷을 'YYYY-MM-DD'로 검색한다.**

```
SELECT *, TO_CHAR(BIRTHDAY, 'YYYY-MM-DD') FROM STUDENT;
```

**8) 학생 테이블에서 '73'년도에 태어난 학생들의 모든 칼럼을 검색한다.**

```
SELECT * FROM STUDENT WHERE TO_CHAR(BIRTHDAY, 'YY') = '73';
```

**9) 학생 테이블에서 '10'월에 태어나지 않은 학생들의 모든 칼럼을 검색한다.**

```
SELECT * FROM STUDENT WHERE TO_CHAR(BIRTHDAY, 'MM') != '10';
```

```
SELECT * FROM STUDENT WHERE TO_CHAR(BIRTHDAY, 'MM') <> '10';
```

**10) 학생 테이블에서 성이 '이'인 학생들의 모든 칼럼을 검색한다.**

```
SELECT * FROM STUDENT WHERE SNAME LIKE '이%';
```

**11) 강좌 테이블에서 강좌이름에 '건축'이 들어가는 강좌들의 모든 칼럼을 검색한다.**

```
SELECT * FROM COURSE WHERE LNAME LIKE '%건축%';
```

- **학사관리 데이터베이스 조인(JOIN)**

1) 교수 테이블에 '510', '강감찬', '건축' 데이터를 입력한다.

```
INSERT INTO PROFESSOR(PCODE, PNAME, DEPT, HIREDATE, SALARY) VALUES('510', '강감찬', '건축', '95/10/01', 2500000);
```

2) 학생 테이블에 '97454005', '홍길동', '건축' 데이터를 입력한다.

```
INSERT INTO STUDENT(SCODE, SNAME, DEPT) VALUES('97454005' ,'홍길동', '전산');
```

3) 학생 테이블 모든 학생의 모든 칼럼과 각 학생의 지도 교수이름, 교수의 학과 칼럼을 검색한다.

```
SELECT S.*, PNAME, P.DEPT
FROM STUDENT S
INNER JOIN PROFESSOR P ON ADVISOR=PCODE;
```

```
SELECT S.*, PNAME, P.DEPT
FROM STUDENT S
LEFT JOIN PROFESSOR P ON ADVISOR=PCODE;
```

```
SELECT S.*, PNAME, P.DEPT
FROM STUDENT S
RIGHT JOIN PROFESSOR P ON ADVISOR=PCODE;
```

4) 강좌 테이블에 'A10', '건축학개론' 데이터를 입력한다.

```
INSERT INTO COURSE(LCODE, LNAME) VALUES('A110', '건축학개론');
```

5) 강좌 테이블 모든 강좌의 모든 칼럼과 각 교수의 담당 교수이름, 교수의 학과 칼럼을 검색한다.

```
SELECT C.*, PNAME
FROM COURSE C
INNER JOIN PROFESSOR ON INSTRUCTOR=PCODE;
```

```
SELECT C.*, PNAME
FROM COURSE C
LEFT JOIN PROFESSOR ON INSTRUCTOR=PCODE;
```

```
SELECT C.*, PNAME
FROM COURSE C
RIGHT JOIN PROFESSOR ON INSTRUCTOR=PCODE;
```

6) 강좌 테이블 모든 강좌의 강좌번호, 강좌이름과 각 강좌를 수강신청 한 학생들의 학번, 수강신청일, 점수 칼럼을 검색한다.

```
SELECT C.LCODE, C.LNAME, E.SCODE, E.GRADE, E.EDATE
FROM COURSE C
LEFT JOIN ENROLLMENT E ON C.LCODE=E.LCODE;
```

7) 학생 테이블 모든 학생의 학생번호, 학생이름과 각 학생이 수강신청 한 강좌들의 강좌번호, 수강신청일, 점수 칼럼을 검색한다.

```
SELECT C.LCODE, C.LNAME, E.SCODE, E.GRADE, E.EDATE
FROM COURSE C
LEFT JOIN ENROLLMENT E ON C.LCODE=E.LCODE;
```

7) 수강신청 테이블 모든 수강신청 데이터의 강좌번호, 강좌이름, 학생번호, 학생이름, 수강신청일, 점수 칼럼을 검색한다.

```
SELECT E.LCODE, LNAME, S.SCODE, SNAME, GRADE, EDATE
FROM ENROLLMENT E
LEFT JOIN COURSE C ON E.LCODE=C.LCODE
LEFT JOIN STUDENT S ON E.SCODE=S.SCODE;
```

- **학사관리 데이터베이스 집계함수**

**1) 교수 테이블의 교수 인원수를 검색한다.**

```
SELECT COUNT(*) 인원수
FROM PROFESSOR;
```

**2) 교수 테이블의 최소 급여, 최대 급여액을 검색한다.**

```
SELECT MIN(SALARY) 최소급여, MAX(SALARY) 최대급여
FROM PROFESSOR;
```

**3) 학생 테이블에서 '전산'과 학생들의 인원수를 검색한다.**

```
SELECT COUNT(*)
FROM STUDENT
WHERE DEPT='전산';
```

**4) 학생 테이블의 학생들은 몇 개 학과에 소속되어있는지 검색한다.**

```
SELECT COUNT(DISTINCT(DEPT)) 학과수
FROM STUDENT;
```

**5) 교수 테이블의 교수인원, 급여평균, 급여합계를 검색한다.**

```
SELECT COUNT(*) 교수인원, AVG(SALARY) 급여평균, SUM(SALARY)
FROM PROFESSOR;
```

```
SELECT  COUNT(*) 교수인원, TO_CHAR(AVG(SALARY),'99,999,999') 급여평균, TO_CHAR(SUM(SALARY),'999,999,999') 급여합계
FROM PROFESSOR;
```

**6) 학생 테이블에서 학과별 학생인원을 검색한다.**

```
SELECT DEPT, COUNT(*) 학생인원
FROM STUDENT
GROUP BY DEPT;
```

**7) 학생 테이블에서 학과, 학년별 학생인원을 학과, 학년별로 오름차순 정렬하여 검색한다.**

```
SELECT DEPT, YEAR, COUNT(*) 학생인원
FROM STUDENT
GROUP BY DEPT, YEAR
ORDER BY DEPT, YEAR;
```

**8) 학생 테이블에서 지도교수별 지도학생수를 검색한다.**

```
SELECT ADVISOR, COUNT(*) 지도학생수
FROM STUDENT
GROUP BY ADVISOR;
```

```
SELECT ADVISOR, PNAME, COUNT(*) 지도학생수
FROM STUDENT
LEFT JOIN PROFESSOR ON ADVISOR=PCODE
GROUP BY ADVISOR, PNAME;
```

**9) 강좌 테이블에서 담당교수별 담당과목수를 검색한다.**

```
SELECT INSTRUCTOR, COUNT(*) 담당과목수
FROM COURSE
GROUP BY INSTRUCTOR;
```

```
SELECT INSTRUCTOR, PNAME, COUNT(*) 담당과목수
FROM COURSE
LEFT JOIN PROFESSOR ON INSTRUCTOR=PCODE
GROUP BY INSTRUCTOR, PNAME;
```

10) 수강신청 테이블에서 수강신청일별 수강인원수를 검색한다.

```
SELECT TO_CHAR(EDATE, 'YYYY-MM-DD') 수강신청일, COUNT(*) 수강인원수
FROM ENROLLMENT
GROUP BY EDATE;
```

11) 수강신청 테이블에서 학생별 학생이름, 평균점수, 합계점수, 수강신청한 과목수를 검색한다.

```
SELECT SCODE, TO_CHAR(AVG(GRADE), '999.99') 평균, SUM(GRADE) 합계, COUNT(*) 과목수
FROM ENROLLMENT
GROUP BY SCODE;
```

```
SELECT E.SCODE, SNAME, TO_CHAR(AVG(GRADE), '999.99') 평균, SUM(GRADE) 합계, COUNT(*) 과목수
FROM ENROLLMENT E
LEFT JOIN STUDENT S ON E.SCODE=S.SCODE
GROUP BY E.SCODE, SNAME;
```

```
SELECT E.SCODE, SNAME, TO_CHAR(AVG(GRADE), '999.99') 평균, SUM(GRADE) 합계, COUNT(*) 과목수
FROM ENROLLMENT E
LEFT JOIN STUDENT S ON E.SCODE=S.SCODE
GROUP BY E.SCODE, SNAME
HAVING AVG(GRADE) >= 75;
```

```
SELECT E.SCODE, SNAME, TO_CHAR(AVG(GRADE), '999.99') 평균, SUM(GRADE) 합계, COUNT(*) 과목수
FROM ENROLLMENT E
LEFT JOIN STUDENT S ON E.SCODE=S.SCODE
GROUP BY E.SCODE, SNAME
HAVING AVG(GRADE) >= 75
ORDER BY AVG(GRADE) DESC;
```

12) 수강신청 테이블에서 강좌별 강좌이름, 평균점수, 합계점수, 수강 신청한 인원수를 검색한다.

```
SELECT LCODE, TO_CHAR(AVG(GRADE), '999.99') 총점, SUM(GRADE) 합계, COUNT(*) 인원수
FROM ENROLLMENT
GROUP BY LCODE;
```

```
SELECT E.LCODE, LNAME, TO_CHAR(AVG(GRADE), '999.99') 총점, SUM(GRADE) 합계, COUNT(*) 인원수
FROM ENROLLMENT E
LEFT JOIN COURSE C ON E.LCODE=C.LCODE
GROUP BY E.LCODE, LNAME;
```

```
SELECT E.LCODE, LNAME, TO_CHAR(AVG(GRADE), '999.99') 총점, SUM(GRADE) 합계, COUNT(*) 인원수
FROM ENROLLMENT E
LEFT JOIN COURSE C ON E.LCODE=C.LCODE
GROUP BY E.LCODE, LNAME
HAVING AVG(GRADE) >= 80;
```

```
SELECT E.LCODE, LNAME, TO_CHAR(AVG(GRADE), '999.99') 총점, SUM(GRADE) 합계, COUNT(*) 인원수
FROM ENROLLMENT E
LEFT JOIN COURSE C ON E.LCODE=C.LCODE
GROUP BY E.LCODE, LNAME
HAVING AVG(GRADE) >= 80
ORDER BY AVG(GRADE) DESC;
```

- **학사관리 데이터베이스 서브쿼리(SUB QUERY)**

**1) 교수 테이블에서 전체 평균 급여보다 많이 받은 교수의 모든 칼럼을 검색한다.**

```
SELECT * FROM PROFESSOR
WHERE SALARY > (SELECT AVG(SALARY) FROM PROFESSOR);
```

**2) 급여(SALARY)을 가장 많이 받는 교수의 모든 칼럼을 검색한다.**

```
SELECT * FROM PROFESSOR
WHERE SALARY = (SELECT MAX(SALARY) FROM PROFESSOR);
```

**3) 강좌이름에 '건축'이 포함된 강좌를 강의하는 교수들의 모든 칼럼을 검색한다.**

```
SELECT * FROM PROFESSOR
WHERE PCODE IN (SELECT INSTRUCTOR FROM COURSE WHERE LNAME LIKE '%건축%');
```

**4) 수강생이 한 명도 없는 강의의 모든 칼럼을 검색한다.**

```
SELECT * FROM COURSE
WHERE LCODE NOT IN (SELECT DISTINCT LCODE FROM ENROLLMENT);
```

**5) 학생 테이블의 모든 칼럼과 교수 테이블의 지도교수 이름을 검색한다.**

```
SELECT S.*, (SELECT PNAME FROM PROFESSOR WHERE PCODE=S.ADVISOR) 지도교수명
FROM STUDENT S;
```

**6) 학생 테이블의 모든 칼럼과 학생별 수강 신청한 강좌수를 검색한다.**

```
SELECT S.*, (SELECT COUNT(*) FROM ENROLLMENT E WHERE E.SCODE=S.SCODE) 강좌수
FROM STUDENT S;
```

**7) 강좌 테이블의 모든 칼럼과 강좌별 강좌평균을 검색한다.**

```
SELECT C.*, (SELECT AVG(GRADE) FROM ENROLLMENT E WHERE E.LCODE=C.LCODE) 평균
FROM COURSE C;
```

**6) 수강신청 테이블의 강좌별 강좌번호, 인원, 평균을 계산한 후 인원이 3명 이상인 강좌를 검색한다.**

```
SELECT T.*
FROM (SELECT LCODE, COUNT(*) 인원, AVG(GRADE) 평균 FROM ENROLLMENT GROUP BY LCODE) T;
```

```
SELECT C.*, T.인원, T.평균
FROM (SELECT LCODE, COUNT(*) 인원, AVG(GRADE) 평균 FROM ENROLLMENT GROUP BY LCODE) T
RIGHT JOIN COURSE C ON C.LCODE=T.LCODE
WHERE T.인원 >= 3;
```

**7) 수강신청 테이블의 학생별 학생번호, 강좌수, 평균을 계산한 후 평균이 75점 이상인 학생을 검색한다.**

```
SELECT T.*
FROM (SELECT SCODE, COUNT(*) 강좌수, AVG(GRADE) 평균 FROM ENROLLMENT GROUP BY SCODE) T;
```

```
SELECT S.*, T.강좌수, T.평균
FROM (SELECT SCODE, COUNT(*) 강좌수, AVG(GRADE) 평균 FROM ENROLLMENT GROUP BY SCODE) T
RIGHT JOIN STUDENT S ON T.SCODE=S.SCODE
WHERE T.평균 >= 75;
```

- **학사관리 데이터베이스 갱신(INSERT, UPDATE, DELETE)**

**1) 교수 테이블에 아래 두 명의 교수 데이터를 입력한다.**

```
INSERT INTO PROFESSOR(PCODE, PNAME, DEPT) VALUES('222', '이교수', '전산');
```

```
INSERT INTO PROFESSOR(PCODE, PNAME, DEPT) VALUES('510', '김교수', '건축');
```

**2) 학생 테이블에 아래 두 명의 학생 데이터를 입력한다.**

```
INSERT INTO STUDENT(SCODE, SNAME, DEPT) VALUES('92414030', '이학생', '전산');
```

```
INSERT INTO STUDENT(SCODE, SNAME, DEPT) VALUES('92414031', '김학생', '전산');
```

**3) 강좌 테이블에 아래 두 개의 강좌 데이터를 입력한다.**

```
INSERT INTO COURSE(LCODE, LNAME) VALUES('C501', '자바 프로그래밍');
```

```
INSERT INTO COURSE(LCODE, LNAME) VALUES('C601', '파이썬 프로그래밍');
```

**4) 수강신청 테이블에 아래 두 명의 수강신청 데이터를 입력한다.**

```
INSERT INTO ENROLLMENT(LCODE, SCODE) VALUES('C401' ,'92414030');
INSERT INTO ENROLLMENT(LCODE, SCODE) VALUES('C501' ,'92414030');
INSERT INTO ENROLLMENT(LCODE, SCODE) VALUES('C601' ,'92414030');
```

```
INSERT INTO ENROLLMENT(LCODE, SCODE) VALUES('C401' ,'92414031');
INSERT INTO ENROLLMENT(LCODE, SCODE) VALUES('C501' ,'92414031');
INSERT INTO ENROLLMENT(LCODE, SCODE) VALUES('C601' ,'92414031');
```

**5) 교수 테이블에서 교수직급이 NULL이면 '시강강사'로 수정한다.**

```
UPDATE PROFESSOR
SET TITLE='시간강사'
WHERE TITLE IS NULL;
```

**6) 교수 테이블에서 '전산'과 교수들의 급여를 10% 증가하여 수정한다.**

```
UPDATE PROFESSOR
SET SALARY=SALARY + (SALARY * 0.1)
WHERE DEPT='전산';
```

**7) 학생 테이블에서 92학번의 학년을 4학년으로 수정한다.**

```
UPDATE STUDENT
SET YEAR=4
WHERE SUBSTR(SCODE, 1, 2) = '92'
```

**8) 학생 테이블에서 '이재광'(228) 교수가 지도하는 학생들의 지도교수를 '이병렬'(221) 교수로 수정한다.**

```
UPDATE STUDENT
SET ADVISOR=(SELECT PCODE FROM PROFESSOR WHERE PNAME='이병렬')
WHERE ADVISOR=(SELECT PCODE FROM PROFESSOR WHERE PNAME='이재광');
```

9) 학생 테이블에서 지도교수가 NULL인 학생들의 지도교수를 '이교수'로 수정한다.

```
UPDATE STUDENT
SET ADVISOR = (SELECT PCODE FROM PROFESSOR WHERE PNAME='이교수')
WHERE ADVISOR IS NULL;
```

10) 강좌 테이블에서 담당교수가 NULL인 강좌들의 담당교수를 '김교수'로 수정한다.

```
UPDATE COURSE
SET INSTRUCTOR=(SELECT PCODE FROM PROFESSOR WHERE PNAME='김교수')
WHERE INSTRUCTOR IS NULL;
```

11) '김교수' 교수가 강의하는 강좌들의 강의실을 모두 '404'호로 수정한다.

```
UPDATE COURSE
SET ROOM='403'
WHERE INSTRUCTOR IN (SELECT PCODE FROM PROFESSOR WHERE PNAME='김교수');
```

12) 수강신청 테이블에서 '파일처리론' 점수가 50점 미만이면 가산점 10점을 부여하여 수정한다.

```
UPDATE ENROLLMENT
SET GRADE=GRADE + 10
WHERE LCODE IN (SELECT LCODE FROM COURSE WHERE LNAME='파일처리론') AND GRADE < 50;
```

13) '파일처리론' 강좌를 수강 신청한 인원을 강좌 테이블의 수강인원으로 수정한다.

```
UPDATE COURSE C
SET PERSONS=(SELECT COUNT(*) FROM ENROLLMENT E WHERE C.LCODE=E.LCODE)
WHERE LNAME='파일처리론';
```

14) 수강신청 테이블에서 강좌별 수강인원을 강좌 테이블의 수강인원으로 수정한다.

```
UPDATE COURSE C
SET PERSONS=(SELECT COUNT(*) FROM ENROLLMENT E WHERE E.LCODE=C.LCODE)
```

15) 수강신청 테이블에서 강좌명이 '프로그래밍'으로 끝나는 강좌를 수강 신청한 학생들을 삭제한다.

```
DELETE FROM ENROLLMENT
WHERE LCODE IN (SELECT LCODE FROM COURSE WHERE LNAME LIKE '%프로그래밍');
```

16) 수강신청 테이블에서 '이학생', '김학생'이 수강 신청한 강좌들을 삭제한다.

```
DELETE FROM ENROLLMENT
WHERE SCODE = (SELECT SCODE FROM STUDENT WHERE SNAME IN ('김학생', '이학생'));
```

17) 학생 테이블에서 학생이름 '김학생'인 학생을 삭제한다.

```
DELETE FROM STUDENT WHERE SNAME IN ('이학생', '김학생');
```

18) 강좌 테이블에서 강좌명이 '프로그래밍'으로 끝나는 강좌를 삭제한다.

```
DELETE FROM COURSE WHERE LNAME LIKE '%프로그래밍';
```

19) 교수 테이블에서 직급이 '시간강사'인 교수들을 삭제한다.

```
DELETE FROM PROFESSOR WHERE TITLE = '시간강사';
```

- **학사관리 데이터베이스 테이블 생성, 제거, 변경**

**1) 수강신청 테이블에서 '김창덕' 학생이 신청한 수강신청 데이터를 새로운 'ENROLL_KIM' 테이블로 생성한다.**

```
CREATE TABLE ENROLL_KIM AS
SELECT * FROM ENROLLMENT WHERE SCODE = (SELECT SCODE FROM STUDENT WHERE SNAME='김창덕');
```

**2) 학생 테이블에서 4학년 학생 데이터를 새로운 'STUDENT_4' 테이블로 생성한다.**

```
CREATE TABLE STUDENT_4 AS
SELECT * FROM STUDENT WHERE YEAR=4;
```

**3) 위에서 생성한 STUDENT_4 테이블에 PRIMARY KEY, FOREIGN KEY를 설정하고 제거한다.**

```
ALTER TABLE STUDENT_4
ADD CONSTRAINT PK_STUDENT PRIMARY KEY(SCODE);
```

```
ALTER TABLE STUDENT_4
ADD CONSTRAINT FK_PROFESSOR FOREIGN KEY(ADVISOR) REFERENCES PROFESSOR(PCODE);
```

```
ALTER TABLE STUDENT_4 DROP PRIMARY KEY;
```

```
ALTER TABLE STUDENT_4 DROP CONSTRAINT FK_PROFESSOR;
```

**4) 위에서 생성한 'ENROLL_KIM' 테이블과 'STUDENT_4' 테이블을 제거한다.**

```
DROP TABLE ENROLL_KIM;
```

```
DROP TABLE STUDENT_4;
```

**5) 교수 테이블에 지도학생 수('STU_CNT')를 저장할 칼럼을 추가하고 학생 테이블에서 학생 수를 구아여 수정한다.**

```
ALTER TABLE PROFESSOR ADD STU_CNT INT DEFAULT 0;
```

```
UPDATE PROFESSOR
SET STU_CNT=(SELECT COUNT(*) FROM STUDENT WHERE ADVISOR=PCODE);
```

**6) 교수 테이블 학과 칼럼의 문자 길이를 35자로, 기본 값을 '전산'으로 변경한다.**

```
ALTER TABLE PROFESSOR MODIFY DEPT VARCHAR(35) DEFAULT '전산';
```

**7) 교수 테이블 'STU_CNT' 칼럼 이름을 'STU_COUNT'로 변경한다.**

```
ALTER TABLE PROFESSOR RENAME COLUMN STU_CNT TO STU_COUNT;
```

**8) 교수 테이블 'STU_COUNT' 칼럼을 삭제한다.**

```
ALTER TABLE PROFESSOR DROP COLUMN STU_CNT;
```

**9) 수강신청 테이블에 입력할 때마다 1씩 증가하는 칼럼('SEQ_ID')을 추가한다.**

```
ALTER TABLE ENROLLMENT ADD SEQ_ID INT GENERATED ALWAYS AS IDENTITY;
```

- **학사관리 데이터베이스 뷰(VIEW)**

**VIEW를 생성하고 생성된 VIEW를 확인하고 생성된 VIEW를 삭제하는 SQL문은 아래와 같다.**

```
CREATE 뷰이름 AS (SELECT 문장);  -- VIEW 생성

SELECT * FROM USER_VIEWS;  -- 생성된 VIEW 확인

DROP VIEW 뷰이름  -- 생성된 VIEW 삭제
```

**1) 학생 테이블 모든 칼럼에 지도 교수이름을 추가하는 VIEW를 생성한다.**

```
INSERT INTO STUDENT(SCODE, SNAME, DEPT) VALUES('92414030', '김학생', '전산');
```

```
CREATE VIEW VIEW_STU AS
SELECT S.*, PNAME
FROM STUDENT S
LEFT JOIN PROFESSOR ON ADVISOR=PCODE;
```

```
INSERT INTO STUDENT(SCODE, SNAME, DEPT) VALUES('92414031', '박학생', '전산');
```

**2) 강좌 테이블 모든 칼럼에 담당 교수이름을 추가하는 VIEW를 생성한다.**

```
INSERT INTO COURSE(LCODE, LNAME) VALUES('C501', '자바 프로그래밍');
```

```
CREATE VIEW VIEW_COU AS
SELECT C.*, PNAME
FROM COURSE C
LEFT JOIN PROFESSOR ON INSTRUCTOR=PCODE;
```

```
INSERT INTO COURSE(LCODE, LNAME) VALUES('C601', '파이썬 프로그래밍');
```

**3) 수강신청 테이블 모든 칼럼에 강좌 이름을 추가하는 VIEW를 생성한다.**

```
CREATE VIEW VIEW_ENROLL_COU AS
SELECT E.LCODE, LNAME, E.SCODE, GRADE, EDATE
FROM ENROLLMENT E
LEFT JOIN COURSE C ON E.LCODE=C.LCODE
ORDER BY E.LCODE, E.SCODE;
```

**4) 수강신청 테이블 모든 칼럼에 학생 이름을 추가하는 VIEW를 생성한다.**

```
CREATE VIEW VIEW_ENROLL_STU AS
SELECT E.SCODE, SNAME, E.LCODE, GRADE, EDATE
FROM ENROLLMENT E
LEFT JOIN STUDENT S ON E.SCODE=S.SCODE
ORDER BY E.SCODE, E.LCODE;
```

**5) 수강신청 테이블 모든 칼럼에 강좌 이름과 학생 이름을 추가하는 VIEW를 생성한다.**

```
CREATE VIEW ENROLL_COU_STU AS
SELECT E.LCODE, LNAME, S.SCODE, SNAME, GRADE, EDATE
FROM ENROLLMENT E
LEFT JOIN COURSE C ON E.LCODE=C.LCODE
LEFT JOIN STUDENT S ON E.SCODE=S.SCODE
ORDER BY E.LCODE, E.SCODE;
```

6) 교수 테이블 모든 칼럼에 지도학생수, 담당강좌수를 추가하는 VIEW를 생성한다.

```
CREATE VIEW VIEW_PRO2 AS
SELECT P.*,
(SELECT COUNT(*) FROM STUDENT WHERE ADVISOR=PCODE) AS 지도학생수,
(SELECT COUNT(*) FROM COURSE WHERE INSTRUCTOR=PCODE) AS 담당강좌수
FROM PROFESSOR P;
```

7) 학생 테이블 모든 칼럼에 수강신청강좌 수, 학생 평균점수를 추가하는 VIEW를 생성한다.

```
CREATE VIEW VIEW_STU2 AS
SELECT S.*,
(SELECT COUNT(*) FROM ENROLLMENT E WHERE E.SCODE=S.SCODE) AS 강좌수,
(SELECT TO_CHAR(AVG(GRADE), '999.99') FROM ENROLLMENT E WHERE E.SCODE=S.SCODE) AS 평균
FROM STUDENT S;
```

8) 강좌 테이블 모든 칼럼에 수강신청학생 수, 강좌 평균점수를 추가하는 VIEW를 생성한다.

```
CREATE VIEW VIEW_COU2 AS
SELECT C.*,
(SELECT COUNT(*) FROM ENROLLMENT E WHERE C.LCODE=E.SCODE) AS 수강인원,
(SELECT TO_CHAR(AVG(GRADE), '999.99') FROM ENROLLMENT E WHERE E.LCODE=C.LCODE) AS 평균
FROM COURSE C;
```

9) 수강신청 테이블에서 수강 신청일별 수강신청일, 수강인원을 출력하는 VIEW를 생성한다.

```
CREATE VIEW VIEW_ENROLL AS
SELECT EDATE, COUNT(*) 수강인원
FROM ENROLLMENT
GROUP BY EDATE;
```

10) 수강신청 테이블에서 강좌별 강좌번호, 강좌이름, 수강 신청한 인원, 평균을 출력하는 VIEW를 생성한다.

```
CREATE VIEW VIEW_ENROLL2 AS
SELECT E.LCODE, LNAME, COUNT(*) 수강인원, AVG(GRADE) 평균
FROM ENROLLMENT E
LEFT JOIN COURSE C ON C.LCODE=E.LCODE
GROUP BY E.LCODE, LNAME;SELECT E.LCODE;
```

11) 수강신청 테이블에서 학생별 학생번호, 학생이름, 수강 신청한 강좌수, 평균을 추가하는 VIEW를 생성하시오.

```
CREATE VIEW VIEW_ENROLL3 AS
SELECT E.SCODE, SNAME, COUNT(*) 강좌수, AVG(GRADE) 평균
FROM ENROLLMENT E
LEFT JOIN STUDENT S ON E.SCODE=S.SCODE
GROUP BY E.SCODE, SNAME;
```

12) 수강신청 테이블에서 학과별, 학년별 학과, 학년, 평균점수를 추가하는 VIEW를 생성한다.

```
CREATE VIEW VIEW_ENROLL4 AS
SELECT DEPT, YEAR, TO_CHAR(AVG(GRADE), '999.99') 평균
FROM ENROLLMENT E
LEFT JOIN STUDENT S ON E.SCODE=S.SCODE
GROUP BY DEPT, YEAR
ORDER BY DEPT, YEAR;
```

# 3. 쇼핑몰 데이터베이스

- **사용자(MEMBER) 테이블의 구조**

| 열이름 | 데이터형 | 정보내용 |
|---|---|---|
| USER_ID | VARCHAR(50) | 사용자 아이디(PK) |
| PASS | VARCHAR(200) | 사용자 비밀번호 |
| USER_NAME | VARCHAR(50) | 사용자 이름 |
| EMAIL | VARCHAR(100) | 사용자 이메일 |
| REG_DATE | DATE | 사용자 등록일 |

- **카테고리(CATEGORY) 테이블의 구조**

| 열이름 | 데이터형 | 정보내용 |
|---|---|---|
| CAT_ID | VARCHAR(50) | 카테고리 아이디(PK) |
| CAT_NAME | VARCHAR(200) | 카테고리 이름 |

- **상품(PRODUCT) 테이블의 구조**

| 열이름 | 데이터형 | 정보내용 |
|---|---|---|
| PROD_ID | INT | 상품 아이디(PK) |
| PROD_NAME | VARCHAR(200) | 상품이름 |
| CAT_ID | VARCHAR(50) | 카테고리 아이디(FK) |
| PRICE | INT | 상품가격 |
| STOCK | INT | 재고수량 |
| REG_DATE | DATE | 상품 등록일 |

- **주문정보(ORDERS) 테이블의 구조**

| 열이름 | 데이터형 | 정보내용 |
|---|---|---|
| ORDER_ID | INT | 주문정보 어이디(PK) |
| USER_ID | VARCHAR(50) | 주문자 아이디(FK) |
| ORDER_DATE | DATE | 주문일자 |
| TOTAL_PRICE | INT | 총 결제금액 |
| STATUS | INT | 주문상태 |

- **주문상품(ORDER_DETAIL) 테이블의 구조**

| 열이름 | 데이터형 | 정보내용 |
|---|---|---|
| DETAIL_ID | INT | 주문상품 아이디(PK) |
| ORDER_ID | INT | 주문정보 아이디(FK) |
| QNT | INT | 주문수량(FK) |
| PRICE | INT | 구매단가 |

- **쇼핑몰 테이블 생성**

사용자 테이블 생성

```
CREATE TABLE MEMBER(
    USER_ID         VARCHAR2(50) NOT NULL PRIMARY KEY,
    PASS            VARCHAR2(200) NOT NULL,
    USER_NAME       VARCHAR2(50) NOT NULL,
    PHONE           VARCHAR2(50),
    ADDR_BASIC      VARCHAR2(200),
    ADDR_DETAIL     VARCHAR2(200),
    EMAIL           VARCHAR2(50),
    REG_DATE        DATE DEFAULT SYSDATE
);
```

카테고리 테이블 생성

```
CREATE TABLE CATEGORY(
    CAT_ID          VARCHAR2(20) NOT NULL,
    CAT_NAME        VARCHAR2(200) NOT NULL,
    REG_DAGE        DATE DEFAULT SYSDATE,
    PRIMARY KEY(CAT_ID)
);
```

상품 테이블 생성

```
CREATE TABLE PRODUCT(
    PROD_ID         INT NOT NULL PRIMARY KEY,
    CAT_ID          VARCHAR(20) NOT NULL,
    PROD_NAME       VARCHAR2(200) NOT NULL,
    PRICE           INT DEFAULT 0,
    STOCK           INT DEFAULT 100,
    REG_DATE        DATE DEFAULT SYSDATE,
    FOREIGN KEY(CAT_ID) REFERENCES CATEGORY(CAT_ID)
);
```

주문정보 테이블 생성

```
CREATE TABLE ORDERS(
    ORDER_ID        INT NOT NULL PRIMARY KEY,
    USER_ID         VARCHAR2(50) NOT NULL,
    RECEIVER        VARCHAR2(50),
    PHONE           VARCHAR2(50),
    ADDR_BASIC      VARCHAR2(200),
    ADDR_DETAIL     VARCHAR2(200),
    TOTAL_PRICE     INT DEFAULT 0,
    ORDER_DATE      DATE DEFAULT SYSDATE,
    STATUS          INT DEFAULT 0,  -- 0: 결제대기중 | 1.결제확인 | 2.배송준비중 | 3.배송중 | 4.배송완료 | 5.주문완료
    FOREIGN KEY(USER_ID) REFERENCES MEMBER(USER_ID)
);
```

주문상품 테이블 생성

```
CREATE TABLE ORDER_DETAIL(
    DETAIL_ID       INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    ORDER_ID        INT NOT NULL,
    PROD_ID         INT NOT NULL,
    QNT             INT DEFAULT 1,
    PRICE           INT DEFAULT 0,
    FOREIGN KEY(ORDER_ID) REFERENCES ORDERS(ORDER_ID),
    FOREIGN KEY(PROD_ID) REFERENCES PRODUCT(PROD_ID)
);
```

- **쇼핑몰 데이터 입력**

**사용자 테이블 데이터입력**

```
--사용자1
INSERT INTO MEMBER(USER_ID, PASS, USER_NAME, PHONE, ADDR_BASIC, ADDR_DETAIL, EMAIL, REG_DATE)
VALUES('red', 'pass', '김철수', '010-1234-5678', '서울 강남구 테헤란로 123', '101동 502호', 'soo@example.com', SYSDATE-20);
--사용자2
INSERT INTO MEMBER(USER_ID, PASS, USER_NAME, PHONE, ADDR_BASIC, ADDR_DETAIL, EMAIL, REG_DATE)
VALUES('blue', 'pass', '이영희', '010-2345-6789', '경기 성남시 분당구 판교역로 45', '203동 1201호', 'hee@example.com', SYSDATE-19);
--사용자3
INSERT INTO MEMBER(USER_ID, PASS, USER_NAME, PHONE, ADDR_BASIC, ADDR_DETAIL, EMAIL, REG_DATE)
VALUES('green', 'pass', '박민수', '010-3456-7890', '부산 해운대구 해운대로 789', '105동 304호', 'soo@example.com', SYSDATE-17);
```

---

**카테고리 테이블 데이터입력**

```
--카테고리1
INSERT INTO CATEGORY(CAT_ID, CAT_NAME) VALUES('ELEC', '전자제품');
--카테고리2
INSERT INTO CATEGORY(CAT_ID, CAT_NAME) VALUES('CLOTH', '의류');
--카테고리3
INSERT INTO CATEGORY(CAT_ID, CAT_NAME) VALUES('SPORTS', '스포츠/레저');
```

---

**상품 테이블 데이터입력**

```
-- 상품번호 자동 생성을 위한 시퀀스
CREATE SEQUENCE SEQ_PROD_ID START WITH 1001 INCREMENT BY 1;

--1001
INSERT INTO PRODUCT(PROD_ID, PROD_NAME, CAT_ID, PRICE, STOCK)
VALUES(SEQ_PROD_ID.NEXTVAL, '울트라 게이밍 노트북', 'ELEC', 1550000, 50);
--1002
INSERT INTO PRODUCT(PROD_ID, PROD_NAME, CAT_ID, PRICE, STOCK)
VALUES(SEQ_PROD_ID.NEXTVAL, '무선 노이즈캔슬링 헤드폰', 'ELEC', 320000, 120);
--1003
INSERT INTO PRODUCT(PROD_ID, PROD_NAME, CAT_ID, PRICE, STOCK)
VALUES(SEQ_PROD_ID.NEXTVAL, '기계식 키보드(적축)', 'ELEC', 125000, 85);
--1004
INSERT INTO PRODUCT(PROD_ID, PROD_NAME, CAT_ID, PRICE, STOCK)
VALUES(SEQ_PROD_ID.NEXTVAL, '오버핏 기모 후드티', 'CLOTH', 59000, 300);
--1005
INSERT INTO PRODUCT(PROD_ID, PROD_NAME, CAT_ID, PRICE, STOCK)
VALUES(SEQ_PROD_ID.NEXTVAL, '슬림핏 데님 팬츠', 'CLOTH', 45000, 250);
--1006
INSERT INTO PRODUCT(PROD_ID, PROD_NAME, CAT_ID, PRICE, STOCK)
VALUES(SEQ_PROD_ID.NEXTVAL, '스마트 워치 시리즈 9', 'ELEC', 499000, 40);
--1007
INSERT INTO product(prod_id, prod_name, cat_id, price, stock)
VALUES(SEQ_PROD_ID.NEXTVAL, '프리미엄 요가 매트', 'SPORTS', 35000, 100);
--1008
INSERT INTO PRODUCT(PROD_ID, PROD_NAME, CAT_ID, PRICE, STOCK)
VALUES(SEQ_PROD_ID.NEXTVAL, '경량 패딩 조끼', 'CLOTH', 89000, 150);
--1009
INSERT INTO PRODUCT(PROD_ID, PROD_NAME, CAT_ID, PRICE, STOCK)
VALUES(SEQ_PROD_ID.NEXTVAL, '4K 업무용 모니터 27인치', 'ELEC', 380000, 30);
--1010
INSERT INTO PRODUCT(PROD_ID, PROD_NAME, CAT_ID, PRICE, STOCK)
VALUES(SEQ_PROD_ID.NEXTVAL, '단백질 쉐이크 보틀', 'SPORTS', 12000, 500);
```

---

**주문정보 테이블 데이터입력**

```
-- 주문정보 번호 자동 생성을 위한 시퀀스
CREATE SEQUENCE SEQ_ORDER_ID START WITH 20260001 INCREMENT BY 1;

--주문 3건
INSERT INTO ORDERS(ORDER_ID, USER_ID, ORDER_DATE, RECEIVER, PHONE, ADDR_BASIC, ADDR_DETAIL)
VALUES(SEQ_ORDER_ID.NEXTVAL, 'red', SYSDATE-6, '김영희', '010-1234-5678', '서울 강남구 테헤란로 123', '101동 502호');
INSERT INTO ORDERS(ORDER_ID, USER_ID, ORDER_DATE, RECEIVER, PHONE, ADDR_BASIC, ADDR_DETAIL)
VALUES(SEQ_ORDER_ID.NEXTVAL, 'blue', SYSDATE-5, '이혜원', '010-2345-6789', '경기 성남시 분당구 판교역로 45', '203동 1201호');
INSERT INTO ORDERS(ORDER_ID, USER_ID, ORDER_DATE, RECEIVER, PHONE, ADDR_BASIC, ADDR_DETAIL)
VALUES(SEQ_ORDER_ID.NEXTVAL, 'green', SYSDATE-5, '박길동', '010-1234-5678', '서울 강남구 테헤란로 123', '101동 502호');
-- 주문 3건
INSERT INTO ORDERS(ORDER_ID, USER_ID, ORDER_DATE, RECEIVER, PHONE, ADDR_BASIC, ADDR_DETAIL)
VALUES(SEQ_ORDER_ID.NEXTVAL, 'blue', SYSDATE-4, '이철수', '010-2345-6789', '경기 성남시 분당구 판교역로 45', '203동 1201호');
INSERT INTO ORDERS(ORDER_ID, USER_ID, ORDER_DATE, RECEIVER, PHONE, ADDR_BASIC, ADDR_DETAIL)
VALUES(SEQ_ORDER_ID.NEXTVAL, 'red', SYSDATE-4, '김미숙', '010-1234-5678', '서울 강남구 테헤란로 123', '101동 502호');
INSERT INTO ORDERS(ORDER_ID, USER_ID, ORDER_DATE, RECEIVER, PHONE, ADDR_BASIC, ADDR_DETAIL)
VALUES(SEQ_ORDER_ID.NEXTVAL, 'green', SYSDATE-3, '박민수', '010-1234-5678', '서울 강남구 테헤란로 123', '101동 502호');
-- 주문 3건
INSERT INTO ORDERS(ORDER_ID, USER_ID, ORDER_DATE, RECEIVER, PHONE, ADDR_BASIC, ADDR_DETAIL)
VALUES(SEQ_ORDER_ID.NEXTVAL, 'red', SYSDATE-3, '김철수', '010-1234-5678', '서울 강남구 테헤란로 123', '101동 502호');
INSERT INTO ORDERS(ORDER_ID, USER_ID, ORDER_DATE, RECEIVER, PHONE, ADDR_BASIC, ADDR_DETAIL)
VALUES(SEQ_ORDER_ID.NEXTVAL, 'blue', SYSDATE-2, '이영희', '010-2345-6789', '경기 성남시 분당구 판교역로 45', '203동 1201호');
INSERT INTO ORDERS(ORDER_ID, USER_ID, ORDER_DATE, RECEIVER, PHONE, ADDR_BASIC, ADDR_DETAIL)
VALUES(SEQ_ORDER_ID.NEXTVAL, 'green', SYSDATE-1, '박미혜', '010-1234-5678', '서울 강남구 테헤란로 123', '101동 502호');
```

**주문상품 테이블 데이터입력**

```
-- 주문 20260001 (4건)
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260001, 1001, 1, 1550000);
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260001, 1002, 1, 320000);
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260001, 1003, 2, 125000);
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260001, 1007, 2, 35000);
-- 주문 20260002 (3건)
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260002, 1004, 2, 59000);
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260002, 1005, 1, 45000);
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260002, 1008, 1, 89000);
-- 주문 20260003 (2건)
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260003, 1003, 1, 125000);
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260003, 1007, 1, 35000);
-- 주문 20260004 (2건)
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260004, 1002, 1, 320000);
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260004, 1003, 1, 125000);
-- 주문 20260005 (3건)
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260005, 1006, 1, 499000);
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260005, 1003, 1, 125000);
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260005, 1008, 1, 89000);
-- 주문 20260006 (2건)
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260006, 1007, 3, 35000);
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260006, 1005, 1, 45000);
-- 주문 20260007 (1건)
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260007, 1008, 3, 89000);
-- 주문 20260008 (4건)
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260008, 1009, 1, 380000);
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260008, 1001, 1, 1550000);
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260008, 1006, 1, 499000);
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260008, 1010, 1, 12000);
-- 주문 20260009 (3건)
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260009, 1010, 1, 12000);
INSERT INTO ORDER_DETAI(ORDER_ID, PROD_ID, QNT, PRICE)L VALUES (20260009, 1007, 1, 35000);
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, QNT, PRICE) VALUES (20260009, 1005, 1, 45000);
```

- **쇼핑몰 데이터베이스 조회**

**1) 상품 테이블의 모든 행, 모든 칼럼의 데이터를 검색한다.**

```
SELECT * FROM PRODUCT;
```

**2) 상품 테이블에서 모든 상품의 상품 아이디, 이름, 가격 칼럼을 검색한다.**

```
SELECT PROD_ID, PROD_NAME, PRICE FROM PRODUCT;
```

**3) 상품 테이블에서 상품들의 카테고리를 중복을 제거하여 검색한다.**

```
SELECT DISTINCT(CAT_ID) FROM PRODUCT;
```

**4) 상품 테이블에서 상품 가격이 50000원 이상이고 100000원 이하인 상품을 검색한다.**

```
SELECT * FROM PRODUCT WHERE PRICE >= 50000 AND PRICE <= 100000;
```

```
SELECT * FROM PRODUCT WHERE PRICE BETWEEN 50000 AND 100000;
```

**5) 상품 테이블에서 카테고리 아이디가 'ELEC'이거나 'SPORTS'인 상품의 모든 칼럼을 검색한다.**

```
SELECT * FROM PRODUCT WHERE CAT_ID='ELEC' OR CAT_ID='SPORTS';
```

```
SELECT * FROM PRODUCT WHERE CAT_ID IN ('ELEC', 'SPORTS');
```

**6) 상품 테이블에서 모든 칼럼과 등록일 포맷을 'YYYY-MM-DD HH24:MI:SS'로 검색한다.**

```
SELECT PRODUCT.*, TO_CHAR(REG_DATE, 'YYYY-MM-DD HH24:MI:SS') FMT_REG_DATE FROM PRODUCT;
```

**7) 상품 테이블에서 '26'년도에 등록한 상품의 모든 칼럼을 검색한다.**

```
SELECT * FROM PRODUCT WHERE TO_CHAR(REG_DATE, 'YY') = '26';
```

**8) 상품 테이블에서 '10'월에 등록하지 않는 상품의 모든 칼럼을 검색한다.**

```
SELECT * FROM PRODUCT WHERE TO_CHAR(REG_DATE, 'MM') != '10';
```

```
SELECT * FROM PRODUCT WHERE TO_CHAR(REG_DATE, 'MM') <> '10';
```

**9) 상품 테이블에서 상품명에 '키보드'가 들어가는 상품의 모든 칼럼을 검색한다.**

```
SELECT * FROM PRODUCT WHERE PROD_NAME LIKE '%키보드%';
```

**10) 집 주소가 '서울'인 사용자들을 검색한다.**

```
SELECT * FROM MEMBER WHERE ADDR_BASIC LIKE '%서울%';
```

**11) 아이디가 'RED'인 사용자 정보를 검색한다.**

```
SELECT * FROM MEMBER WHERE USER_ID = LOWER('RED');
```

```
SELECT * FROM MEMBER WHERE UPPER(USER_ID) = 'RED';
```

- **쇼핑몰 데이터베이스 조인(JOIN)**

1) 새로운 카테고리를 등록한다.

```
INSERT INTO CATEGORY (CAT_ID, CAT_NAME) VALUES ('FOOD', '식품');
```

2) 상품 테이블의 모든 칼럼과 각 카테고리 테이블의 카테고리 이름을 검색한다. (카테고리 이름 '스포츠' 포함)

```
SELECT P.*, CAT_NAME
FROM PRODUCT P
LEFT JOIN CATEGORY C ON P.CAT_ID=C.CAT_ID
ORDER BY PROD_ID;
```

3) 새로운 사용자를 등록한다.

```
INSERT INTO MEMBER(USER_ID, PASS, USER_NAME) VALUES('pink', 'pass', '김하늘');
```

4) 주문정보 테이블의 주문정보와 사용자 테이블의 주문자 이름, 이메일을 검색한다. (주문자 이름 '김철수')

```
SELECT O.ORDER_ID, O.ORDER_DATE, O.USER_ID, M.USER_NAME, M.EMAIL
FROM ORDERS O
LEFT JOIN MEMBER M ON O.USER_ID=M.USER_ID
ORDER BY ORDER_ID;
```

5) 주문정보 테이블의 주문정보와 주문상품 테이블의 상품 아이디, 구매단가를 검색한다.

```
SELECT O.ORDER_ID, O.ORDER_DATE, D.PROD_ID, D.PRICE
FROM ORDERS O
LEFT JOIN ORDER_DETAIL D ON O.ORDER_ID=D.ORDER_ID
ORDER BY O.ORDER_ID;
```

6) 위 검색내용에 상품 테이블의 상품 이름 칼럼을 추가한다. (상품 이름 '헤드폰' 포함)

```
SELECT O.ORDER_ID, O.ORDER_DATE, D.PROD_ID, D.PRICE, P.PROD_NAME
FROM ORDERS O
LEFT JOIN ORDER_DETAIL D ON O.ORDER_ID=D.ORDER_ID
LEFT JOIN PRODUCT P ON D.PROD_ID=P.PROD_ID
ORDER BY O.ORDER_ID;
```

7) 위 검색내용에 사용자 테이블의 주문자 이름 칼럼을 추가한다. (주문자 이름 박민수)

```
SELECT O.ORDER_ID, O.ORDER_DATE, D.PROD_ID, D.PRICE, P.PROD_NAME, M.USER_NAME
FROM ORDERS O
LEFT JOIN ORDER_DETAIL D ON O.ORDER_ID=D.ORDER_ID
LEFT JOIN PRODUCT P ON D.PROD_ID=P.PROD_ID
LEFT JOIN MEMBER M ON O.USER_ID=M.USER_ID
ORDER BY O.ORDER_ID;
```

8) 위 검색내용에 카테고리 테이블의 카테고리 이름 칼럼을 추가한다. (카테고리 이름에 '스포츠' 포함)

```
SELECT O.ORDER_ID, O.ORDER_DATE, D.PROD_ID, D.PRICE, P.PROD_NAME, M.USER_NAME, C.CAT_NAME
FROM ORDERS O
LEFT JOIN ORDER_DETAIL D ON O.ORDER_ID=D.ORDER_ID
LEFT JOIN PRODUCT P ON D.PROD_ID=P.PROD_ID
LEFT JOIN MEMBER M ON O.USER_ID=M.USER_ID
LEFT JOIN CATEGORY C ON P.CAT_ID=C.CAT_ID
ORDER BY O.ORDER_ID;
```

- **쇼핑몰 데이터베이스 집계함수**

**1) 카테고리 테이블에서 카테고리별 상품수를 검색한다. (상품 이름 추가)**

```
SELECT P.CAT_ID, COUNT(*) 상품수
FROM PRODUCT P
GROUP BY P.CAT_ID;
```

```
SELECT P.CAT_ID, CAT_NAME, COUNT(*) 상품수
FROM PRODUCT P
LEFT JOIN CATEGORY C ON P.CAT_ID=C.CAT_ID
GROUP BY P.CAT_ID, CAT_NAME;
```

**2) 주문정보 테이블에서 주문자 아이디별 주문수를 검색한다. (주문자 이름 추가)**

```
SELECT O.USER_ID, COUNT(*) 주문수
FROM ORDERS O
GROUP BY O.USER_ID;
```

```
SELECT O.USER_ID, USER_NAME, COUNT(*) 주문수
FROM ORDERS O
LEFT JOIN MEMBER M ON O.USER_ID=M.USER_ID
GROUP BY O.USER_ID, USER_NAME;
```

**3) 주문상품 테이블에서 상품별 주문상품수, 주문총금액을 검색한다. (상품 이름 추가)**

```
SELECT D.PROD_ID, SUM(QNT) 주문상품수, SUM(QNT*D.PRICE) 주문총금액
FROM ORDER_DETAIL D
GROUP BY D.PROD_ID;
```

```
SELECT D.PROD_ID, PROD_NAME, SUM(QNT) 주문상품수, SUM(QNT*D.PRICE) 주문총금액
FROM ORDER_DETAIL D
LEFT JOIN PRODUCT P ON D.PROD_ID=P.PROD_ID
GROUP BY D.PROD_ID, PROD_NAME;
```

**4) 주문상품 테이블에서 카테고리별 주문상품수, 주문총금액을 검색한다. (카테고리 이름 추가 출력)**

```
SELECT P.CAT_ID, CAT_NAME, SUM(QNT) 주문상품수, SUM(QNT*D.PRICE) 주문총금액
FROM ORDER_DETAIL D
LEFT JOIN PRODUCT P ON P.PROD_ID=D.PROD_ID
LEFT JOIN CATEGORY C ON P.CAT_ID=C.CAT_ID
GROUP BY P.CAT_ID, CAT_NAME;
```

**4) 주문상품 테이블에서 주문일별 주문상품수, 주문총금액을 검색한다.**

```
SELECT TO_CHAR(ORDER_DATE, 'YYYY-MM-DD') 년월, SUM(QNT) 주문상품수, SUM(QNT*D.PRICE) 주문총금액
FROM ORDERS O
LEFT JOIN ORDER_DETAIL D ON O.ORDER_ID=D.ORDER_ID
GROUP BY TO_CHAR(ORDER_DATE, 'YYYY-MM-DD');
```

**5) 주문상품 테이블에서 시도별 주문상품수, 주문총금액을 검색한다.**

```
SELECT SUBSTR(ADDR_BASIC, 1, INSTR(ADDR_BASIC, ' ')) 시도, SUM(QNT) 주문상품수, SUM(D.QNT*PRICE) 주문총금액
FROM ORDERS O
LEFT JOIN ORDER_DETAIL D ON O.ORDER_ID=D.ORDER_ID
GROUP BY SUBSTR(ADDR_BASIC, 1, INSTR(ADDR_BASIC, ' '));
```

- **쇼핑몰 데이터베이스 서브쿼리(SUB QUERY)**

**1) 상품 테이블에서 판매가격이 가장 비싼 상품정보를 검색한다.**

```
SELECT *
FROM PRODUCT
WHERE PRICE  = (SELECT MAX(PRICE) FROM PRODUCT);
```

**2) 상품 테이블에서 카테고리 이름이 '전자제품'인 상품들을 검색한다.**

```
SELECT *
FROM PRODUCT
WHERE CAT_ID=(SELECT CAT_ID FROM CATEGORY WHERE CAT_NAME='전자제품');
```

**3) 주문상품 테이블에서 판매수량이 5개 이하인 상품들을 검색한다.**

```
SELECT *
FROM PRODUCT
WHERE PROD_ID IN (SELECT PROD_ID FROM ORDER_DETAIL GROUP BY PROD_ID HAVING SUM(QNT) < 5);
```

**4) 사용자 테이블의 모든 칼럼과 주문자별 주문횟수를 검색한다.**

```
SELECT M.*,
(SELECT COUNT(*) FROM ORDERS O WHERE M.USER_ID=O.USER_ID) 주문횟수
FROM MEMBER M;
```

**5) 상품 테이블 모든 칼럼과 상품별 주문 수량합계, 주문 총금액을 검색한다.**

```
SELECT P.*,
(SELECT SUM(QNT) FROM ORDER_DETAIL D WHERE P.PROD_ID=D.PROD_ID) 수량합계,
(SELECT SUM(QNT*PRICE) FROM ORDER_DETAIL D WHERE P.PROD_ID=D.PROD_ID) 총금액
FROM PRODUCT P;
```

**6) 카테고리 테이블의 모든 칼럼과 카테고리별 주문 수량합계, 총금액을 검색한다.**

```
SELECT C.*,
(SELECT SUM(QNT) FROM ORDER_DETAIL D JOIN PRODUCT P ON C.CAT_ID=P.CAT_ID GROUP BY C.CAT_ID) 수량합계,
(SELECT SUM(QNT*D.PRICE) FROM ORDER_DETAIL D JOIN PRODUCT P ON C.CAT_ID=P.CAT_ID GROUP BY C.CAT_ID) 총금액
FROM CATEGORY C;
```

**7) 사용자 테이블의 사용자별 모든 칼럼과 주문횟수를 검색한다.**

```
SELECT M.*, T.주문횟수
FROM (SELECT USER_ID, COUNT(*) 주문횟수 FROM ORDERS GROUP BY USER_ID) T
RIGHT JOIN MEMBER M ON T.USER_ID=M.USER_ID;
```

**8) 카테고리 테이블의 카테고리별 모든 칼럼과 상품수량을 검색한다.**

```
SELECT C.*, T.상품수량
FROM (SELECT CAT_ID, COUNT(*) 상품수량 FROM PRODUCT GROUP BY CAT_ID) T
RIGHT JOIN CATEGORY C ON C.CAT_ID=T.CAT_ID;
```

**9) 상품 테이블의 상품별 모든 칼럼과 주문 수량합계, 주문 총금액을 검색한다.**

```
SELECT P.*, T.수량합계, T.총금액
FROM (SELECT PROD_ID, SUM(QNT) 수량합계, SUM(QNT*PRICE) 총금액 FROM ORDER_DETAIL GROUP BY PROD_ID) T
RIGHT JOIN PRODUCT P ON T.PROD_ID=P.PROD_ID;
```

- **쇼핑몰 데이터베이스 갱신(INSERT, UPDATE, DELETE)**

1) 새로운 'BOOK' 카테고리를 추가한다.

```
INSERT INTO CATEGORY(CAT_ID, CAT_NAME) VALUES('BOOK', '도서');
```

2) 새로운 'black' 사용자를 추가한다.

```
INSERT INTO MEMBER(USER_ID, PASS, USER_NAME) VALUES('black', 'pass' , '김블랙');
```

2) 새로운 '자바' 상품을 추가한다.

```
INSERT INTO PRODUCT(PROD_ID, PROD_NAME, CAT_ID, PRICE) VALUES(SEQ_PROD_ID.NEXTVAL, '자바', 'BOOK', '19000');
```

4) 'black' 사용자가 '자바' 상품을 주문한다.

```
INSERT INTO ORDERS(ORDER_ID, USER_ID) VALUES(SEQ_ORDER_ID.NEXTVAL, 'black');
```

```
INSERT INTO ORDER_DETAIL(ORDER_ID, PROD_ID, PRICE, QNT) VALUES(SEQ_ORDER_ID.CURRVAL,
(SELECT PROD_ID FROM PRODUCT WHERE PROD_NAME='자바'), (SELECT PRICE FROM PRODUCT WHERE PROD_NAME='자바', 3));
```

5) '김블랙' 사용자의 정보를 수정한다.

```
UPDATE MEMBER
SET PHONE='010-1010-2020', ADDR_BASIC='서울 강서구 화곡동 현대아파트'
WHERE USER_NAME='김블랙';
```

6) 상품 테이블 '도서' 카테고리 상품들의 재고수량을 10% 추가하여 수정한다. (수량은 소수점 첫째자리에서 반올림)

```
UPDATE PRODUCT
SET STOCK = STOCK + ROUND(STOCK * 0.1)
WHERE CAT_ID = (SELECT CAT_ID FROM CATEGORY WHERE CAT_NAME='도서');
```

7) 주문상품 테이블에서 주문 합계액을 계산하여 주문정보 테이블 주문 합계액을 수정한다.

```
UPDATE ORDERS O
SET TOTAL_PRICE = (SELECT SUM(QNT * PRICE) FROM ORDER_DETAIL D WHERE O.ORDER_ID=D.ORDER_ID);
```

8) 주문상품 테이블에서 주문 수량합계를 계산하여 상품 테이블 재고수량을 수정한다.

```
UPDATE PRODUCT P
SET STOCK = STOCK - (SELECT SUM(QNT) FROM ORDER_DETAIL D WHERE P.PROD_ID=D.PROD_ID);
```

9) 주문정보 테이블에서 'black'이 주문한 주문내역을 삭제한다.

```
DELETE FROM ORDER_DETAIL
WHERE ORDER_ID IN (SELECT ORDER_ID FROM ORDERS WHERE USER_ID='black');
```

```
DELETE FROM ORDERS WHERE USER_ID='blacK';
```

10) 카테고리 테이블에서 '도서' 카테고리를 삭제한다.

```
DELETE FROM PRODUCT
WHERE CAT_ID = (SELECT CAT_ID FROM CATEGORY WHERE CAT_NAME='도서');
```

```
DELETE FROM CATEGORY WHERE CAT_NAME='도서';
```

- **쇼핑몰 데이터베이스 테이블 생성, 제거, 변경**

1) 카테고리 이름이 '전자제품'인 상품들을 PRODUCT_ELEC 테이블로 생성한다.

```
CREATE TABLE PRODUCT_ELEC AS
SELECT * FROM PRODUCT WHERE CAT_ID = (SELECT CAT_ID FROM CATEGORY WHERE CAT_NAME='전자제품');
```

2) 위에서 생성한 PRODUCT_ELEC 테이블에 PRIMARY KEY, FOREIGN KEY를 지정한 후 삭제한다.

```
ALTER TABLE PRODUCT_ELEC
ADD CONSTRAINT PK_PRODUCT PRIMARY KEY(PROD_ID);
```

```
ALTER TABLE PRODUCT_ELEC
ADD CONSTRAINT FK_CATEGORY FOREIGN KEY(CAT_ID) REFERENCES CATEGORY(CAT_ID);
```

```
ALTER TABLE PRODUCT_ELEC DROP PRIMARY KEY;
```

```
ALTER TABLE PRODUCT_ELEC DROP CONSTRAINT FK_CATEGORY;
```

3) 배송지가 '서울'인 주문내역을 ORDERS_SEOUL 테이블로 생성한다.

```
CREATE TABLE ORDERS_SEOUL AS
SELECT * FROM ORDERS WHERE ADDR_BASIC LIKE '서울%';
```

4) 위에서 생성한 CATEGORY_ELEC 테이블과 ORDERS_SEOUL 테이블을 제거한다.

```
DROP TABLE CATEGORY_ELEC;
```

```
DROP TABLE ORDERS_SEOUL;
```

5) 카테고리 테이블에 상품수(PROD_CNT)를 저장할 칼럼을 추가하고 상품 테이블에서 상품수를 구하여 수정한다.

```
ALTER TABLE CATEGORY ADD PROD_CNT INT DEFAULT 0;
```

```
UPDATE CATEGORY C
SET PROD_CNT = (SELECT COUNT(*) FROM PRODUCT P WHERE C.CAT_ID=P.CAT_ID);
```

6) 사용자 테이블 이메일 칼럼의 문자길이와 기본값을 변경한다.

```
ALTER TABLE MEMBER MODIFY EMAIL VARCHAR(100) DEFAULT '';
```

7) 카테고리 테이블 'PROD_CNT' 칼럼 이름을 'PROD_COUNT'로 변경한다.

```
ALTER TABLE CATEGORY RENAME COLUMN PROD_CNT TO PROD_COUNT;
```

8) 카테고리 테이블 'PROD_COUNT' 칼럼을 삭제한다.

```
ALTER TABLE CATEGORY DROP COLUMN PROD_COUNT;
```

9) 사용자 테이블에 입력할 때마다 1씩 증가는 칼럼(SEQ_ID)을 추가한다.

```
ALTER TABLE MEMBER ADD SEQ_ID INT GENERATED ALWAYS AS IDENTITY;
```

- **쇼핑몰 데이터베이스 뷰(VIEW)**

**VIEW를 생성하고 생성된 VIEW를 확인하고 생성된 VIEW를 삭제하는 SQL문은 아래와 같다.**

```
CREATE 뷰이름 AS (SELECT 문장);  -- VIEW 생성

SELECT * FROM USER_VIEWS;  -- 생성된 VIEW 확인

DROP VIEW 뷰이름  -- 생성된 VIEW 삭제
```

**1) 상품 테이블 모든 칼럼에 카테고리명을 추가하는 VIEW를 생성한다.**

```
CREATE VIEW VIEW_PROD AS
SELECT P.*, C.CAT_NAME
FROM PRODUCT P
LEFT JOIN CATEGORY C ON P.CAT_ID = C.CAT_ID;
```

**2) 주문정보 테이블 모든 칼럼에 주문자명을 추가하는 VIEW를 생성한다.**

```
CREATE VIEW VIEW_MEMBER AS
SELECT O.*, M.USER_NAME
FROM ORDERS O
LEFT JOIN MEMBER M ON O.USER_ID=M.USER_ID;
```

**3) 주문상품 테이블에 모든 칼럼에 상품명을 추가하는 VIEW를 생성한다.**

```
CREATE VIEW VIEW_DETAIL AS
SELECT D.*, P.PROD_NAME
FROM ORDER_DETAIL D
LEFT JOIN PRODUCT P ON D.PROD_ID=P.PROD_ID;
```

**4) 주문상품 테이블에 모든 칼럼에 상품명, 주문자명을 추가하는 VIEW를 생성한다.**

```
CREATE VIEW VIEW_DETAIL2 AS
SELECT D.*, P.PROD_NAME, M.USER_NAME
FROM ORDER_DETAIL D
LEFT JOIN PRODUCT P ON D.PROD_ID=P.PROD_ID
LEFT JOIN ORDERS O ON D.ORDER_ID=O.ORDER_ID
LEFT JOIN MEMBER M ON M.USER_ID=O.USER_ID;
```

**5) 주문상품 테이블에 모든 칼럼에 상품명, 주문자명, 카테고리명을 추가하는 VIEW를 생성한다.**

```
CREATE VIEW VIEW_DETAIL3 AS
SELECT D.*, P.PROD_NAME, M.USER_NAME, C.CAT_NAME
FROM ORDER_DETAIL D
LEFT JOIN PRODUCT P ON D.PROD_ID=P.PROD_ID
LEFT JOIN ORDERS O ON D.ORDER_ID=O.ORDER_ID
LEFT JOIN MEMBER M ON M.USER_ID=O.USER_ID
LEFT JOIN CATEGORY C ON P.CAT_ID=C.CAT_ID;
```

**6) 사용자 테이블의 모든 칼럼과 사용자별 주문횟수를 추가하는 VIEW를 생성한다.**

```
CREATE VIEW VIEW_MEMBER2 AS
SELECT M.*,(SELECT COUNT(*) FROM ORDERS O WHERE M.USER_ID=O.USER_ID) 주문횟수
FROM MEMBER M;
```

**7) 카테고리 테이블의 모든 칼럼과 카테고리별 상품수를 추가하는 VIEW를 생성한다.**

```
CREATE VIEW VIEW_PROD2 AS
SELECT C.*, (SELECT COUNT(*) FROM PRODUCT P WHERE C.CAT_ID=P.CAT_ID) 상품수
FROM CATEGORY C;
```

**8) 상품 테이블 모든 칼럼에 상품별 주문 수량합계, 총금액을 추가하는 VIEW를 생성한다.**

```
CREATE VIEW VIEW_PROD3 AS
SELECT P.*,
(SELECT SUM(QNT) FROM ORDER_DETAIL D WHERE P.PROD_ID=D.PROD_ID) 수량합계,
(SELECT SUM(QNT*PRICE) FROM ORDER_DETAIL D WHERE P.PROD_ID=D.PROD_ID) 총금액
FROM PRODUCT P;
```

**9) 카테고리 테이블의 모든 칼럼과 카테고리별 주문 수량합계, 총금액을 추가하는 VIEW를 생성한다.**

```
CREATE VIEW VIEW_CATEGORY AS
SELECT C.*,
(SELECT SUM(QNT) FROM ORDER_DETAIL D JOIN PRODUCT P ON D.PROD_ID=P.PROD_ID WHERE C.CAT_ID=P.CAT_ID) 수량합계,
(SELECT SUM(QNT*D.PRICE) FROM ORDER_DETAIL D JOIN PRODUCT P ON D.PROD_ID=P.PROD_ID WHERE C.CAT_ID=P.CAT_ID)
총금액
FROM CATEGORY C;
```

**10) 사용자 테이블의 모든 칼럼과 사용자별 주문 수량합계, 총금액을 추가하는 VIEW를 생성한다.**

```
CREATE VIEW VIEW_MEMBER3 AS
SELECT M.*,
(SELECT SUM(QNT) FROM ORDER_DETAIL D JOIN ORDERS O ON D.ORDER_ID=O.ORDER_ID WHERE O.USER_ID=M.USER_ID) 수량합계,
(SELECT SUM(QNT*PRICE) FROM ORDER_DETAIL D JOIN ORDERS O ON D.ORDER_ID=O.ORDER_ID WHERE O.USER_ID=M.USER_ID)
총금액
FROM MEMBER M;
```

**11) 주문상품 테이블에서 상품별 상품아이디, 상품명, 주문한 수량합계, 총금액을 추가하는 VIEW를 생성한다.**

```
CREATE VIEW VIEW_DETAIL4 AS
SELECT D.PROD_ID, P.PROD_NAME, SUM(QNT) 수량합계, SUM(QNT*D.PRICE) 총금액
FROM ORDER_DETAIL D
LEFT JOIN PRODUCT P ON D.PROD_ID=P.PROD_ID
GROUP BY D.PROD_ID, P.PROD_NAME;
```

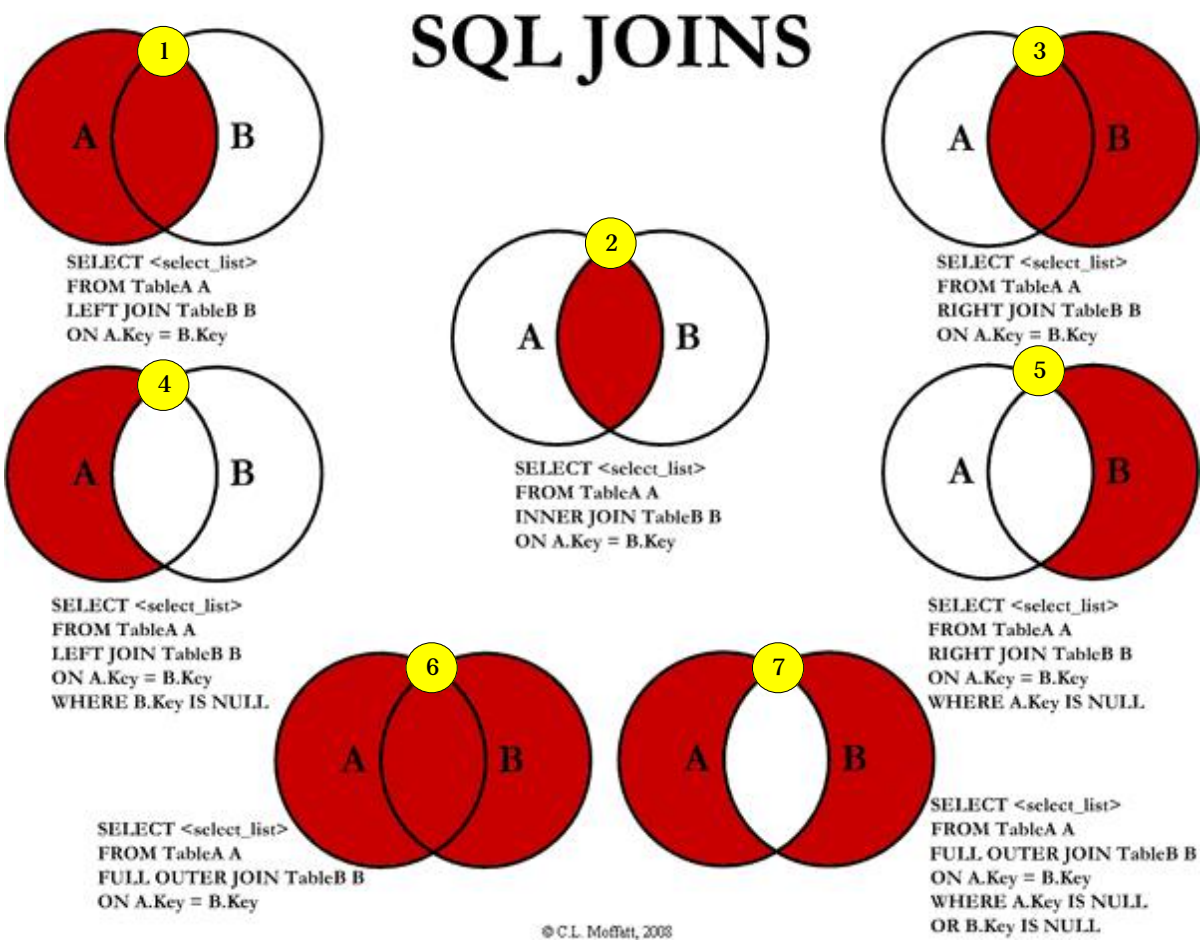
**12) 주문상품 테이블에서 주문일별 주문일, 주문한 수량합계, 총금액을 추가하는 VIEW를 생성한다.**

```
CREATE VIEW VIEW_DETAIL5 AS
SELECT ORDER_DATE, SUM(QNT) 수량합계, SUM(QNT*D.PRICE) 총금액
FROM ORDER_DETAIL D
LEFT JOIN ORDERS O ON D.ORDER_ID=O.ORDER_ID
GROUP BY ORDER_DATE;
```

**13) 주문상품 테이블에서 사용자 아이디별 사용자 아이디, 사용자명, 주문한 수량합계, 총금액을 추가하는 VIEW를 생성한다.**

```
CREATE VIEW VIEW_DETAIL6 AS
SELECT M.USER_ID, M.USER_NAME, SUM(QNT) 수량합계, SUM(QNT*D.PRICE) 총금액
FROM ORDER_DETAIL D
LEFT JOIN ORDERS O ON D.ORDER_ID=O.ORDER_ID
LEFT JOIN MEMBER M ON O.USER_ID=M.USER_ID
GROUP BY M.USER_ID, M.USER_NAME;
```

- **SQL JOINS**

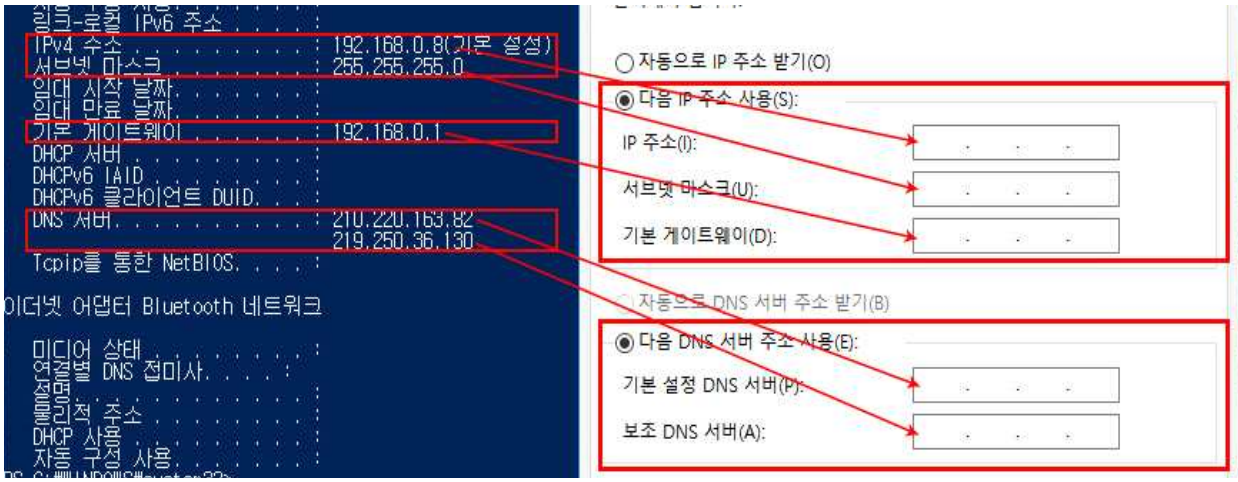



```
INSERT INTO PROFESSOR(PCODE, PNAME, DEPT, HIREDATE) VALUES('100', '심청이', '전산', SYSDATE);
INSERT INTO STUDENT(SCODE, SNAME, DEPT, BIRTHDAY) VALUES('92010001','홍길동','전산', '1969/05/08');

--1
SELECT A.*, PNAME
FROM STUDENT A
LEFT JOIN PROFESSOR B ON A.ADVISOR=B.PCODE;
--2
SELECT A.*, PNAME
FROM STUDENT A
INNER JOIN PROFESSOR B ON A.ADVISOR=B.PCODE;
--3
SELECT A.*, PNAME
FROM STUDENT A
RIGHT JOIN PROFESSOR B ON A.ADVISOR=B.PCODE;
--4
SELECT A.*, PNAME
FROM STUDENT A
LEFT JOIN PROFESSORS B ON A.ADVISOR=B.PCODE
WHERE B.PCODE IS NULL;
--5
SELECT A.*, PNAME
FROM STUDENT A
RIGHT JOIN PROFESSORS B ON A.ADVISOR=B.PCODE
WHERE A.ADVISOR IS NULL;
--6
SELECT A.*, B.*
FROM STUDENT A
FULL OUTER JOIN PROFESSOR B ON A.ADVISOR=B.PCODE;
--7
SELECT A.*, B.*
FROM STUDENT A
FULL OUTER JOIN PROFESSOR B ON A.ADVISOR=B.PCODE
WHERE A.ADVISOR IS NULL OR B.PCODE IS NULL;
```

# • 데이터베이스 서버 외부에서 접속하기 (+IPTIME 설정)

## 1) 고정 아이피 설정




## 2) 내부포트 열기 (방화벽 해제)

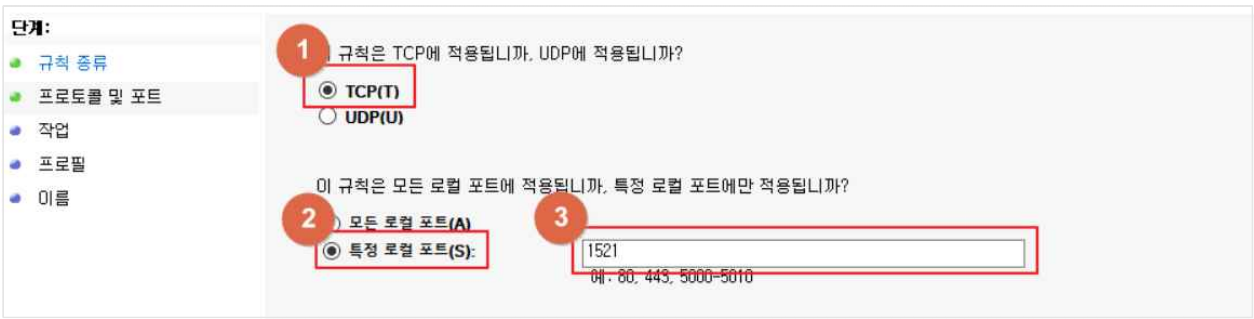
1. [제어판]-[모든 제어판 항목]-[Windows Defender 방화벽]-[고급설정]
2. [인바운드 규칙]-[새 규칙]




3. [포트 선택] - [다음]




4. [TCP] -[특정 로컬 포트] - [다음]



5. [연결 허용 체크] - [다음] - [도메인, 개인, 공용 체크] - [다음] -[이름입력] - [마침]

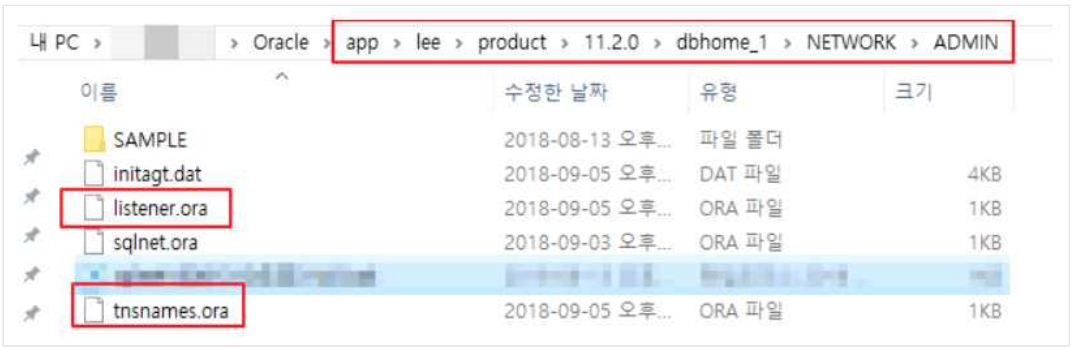



## 3) Oracle 서버 PC의 listener.ora와 tnsnames.ora 설정

1. [바탕화면] - [내PC] -[속성] - [컴퓨터 이름]




2. Oracle이 설치된 위치로 이동 [설치장소]-[app]-[oracle]-[product]-[11.2.0]-[server]-[newtwork]-[ADMIN]




3. listener.org 파일 HOST= localhost라 되어 있는 것을 컴퓨터 이름으로 수정한 후 저장

```
LISTENER =
  (DESCRIPTION_LIST =
    (DESCRIPTION =
      (ADDRESS = (PROTOCOL = IPC)(KEY = EXTPROC1521))
      (ADDRESS = (PROTOCOL = TCP)(HOST = Lee-PC)(PORT = 1521))
    )
  )
```

4. tnsnames.ora 파일도 마찬가지로 HOST=localhost라 쓰여져 있는 것을 컴퓨터 이름으로 수정한 후 저장

```
# tnsnames.ora Network Configuration File: [redacted]product\11.2.0\dbhome_1\network\admin\tnsnames.ora
# Generated by Oracle configuration tools.

LISTENER_MYORACLE =
  (ADDRESS = (PROTOCOL = TCP)(HOST = Lee-PC)(PORT = 1521))
```

```
MYORACLE =
  (DESCRIPTION =
    (ADDRESS = (PROTOCOL = TCP)(HOST = Lee-PC)(PORT = 1521))
    (CONNECT_DATA =
      (SERVER = DEDICATED)
      (SERVICE_NAME = myoracle)
    )
  )
```

- **MySQL 외부 컴퓨터에서 접근 허용**

**MySQLI 8.0인 경우 비밀번호 변경**

```
ALTER USER 'web'@'%'
IDENTIFIED WITH mysql_native_password BY 'pass';
```

**localhost에만 접속 허용**

```
GRANT ALL PRIVILEGES ON webdb.* TO web@'localhost' IDENTIFIED BY 'pass';
```

**192.169.x.x로 시작되는 모든 IP에서 접속 허용**

```
GRANT ALL PRIVILEGES ON webdb.* TO web@'192.168.%' IDENTIFIED BY 'pass';
```

**외부 컴퓨터에서 접속 허용**

```
GRANT ALL PRIVILEGES ON webdb.* TO web@'%' IDENTIFIED BY 'pass';
```

**접속허용 확인**

```
SELECT HOST, USER FROM MYSQL.USER;
```

**접속허용 제거**

```
DELETE FROM MYSQL.USER WHERE HOST='192.168.%' and USER='web';
```

- **외부포트 허용하기 (포트 포워딩:iptime 설정)**

**1. [크롬]-[192.168.0.1]-[로그인]-[관리도구] [cmd]-[ipconfig/all] 기본게이트**




**2. [고급설정]-[NAT/라우터 설정]-[포트포워드 설정]**




**3. 포트 포워드 설정하기**

